

Encoder-Decoder Attention: Full Hand-Computed Worked Example

Translating “Let’s go” $\rightarrow$ “Vamos” with Attention (15 Steps)

Gary Ewing

This document works, by hand, through every arithmetic step of the Attention example used in *The StatQuest Illustrated Guide to Neural Networks and AI* (Josh Starmer), in the section “Attention: Details” (book pages 209–218). It follows the same style as the LSTM forward-pass document: every constant is listed up front, and every single arithmetic operation — every multiplication, every sum, every exponential inside a SoftMax — is written out explicitly, step by step, exactly as numbered in the book (steps 1 through 15).

As in the book, the trained Encoder and Decoder LSTM outputs (the “Short-Term Memories”) are treated as *given* constants rather than recomputed from scratch — the book explicitly notes that the weights and biases behind these particular LSTM outputs are not the same ones used in the earlier chapter’s worked LSTM example, so they cannot be hand-derived from those weights. Everything downstream of those given LSTM outputs — every Similarity Score, every SoftMax, every Attention Value, and every Fully Connected layer decision — is fully derived by hand below.

The Setup

We are translating the two-word English sentence **Let’s go** into Spanish. The Encoder reads the sentence one word at a time; the Decoder then generates the Spanish translation one token at a time, and Attention gives the Decoder direct access to every Encoder output at every Decoder step.

Vocabularies

- Encoder (English) vocabulary, in order: [Let’s, to, go, <EOS>]
- Decoder (Spanish) vocabulary, in order: [ir, Vamos, y, <EOS>]

Constants (Given Encoder/Decoder LSTM Outputs)

- Encoder Short-Term Memory after reading **Let’s**: $h_1^{enc} = -0.73$
- Encoder Short-Term Memory after reading **go**: $h_2^{enc} = -0.68$
- Decoder Short-Term Memory after its first input token <EOS>: $h_1^{dec} = -0.99$
- Decoder Short-Term Memory after its second input token **Vamos**: $h_2^{dec} = 0.76$

Constants (Given Fully Connected Layer Outputs)

The Fully Connected layer’s weights and bias are also pretrained and are not given numerically in the book; only its raw output values (one per vocabulary word) are given, in the same vocabulary order [ir, Vamos, y, <EOS>]:

- Pass 1 raw scores (inputs: Attention Value 1 and h_1^{dec}): [1.7, 2.1, -3, -1]
- Pass 2 raw scores (inputs: Attention Value 2 and h_2^{dec}): [-1, 0, -2, 4.4]

The Two Formulas Used Throughout

Similarity Score (a simple two-number stand-in for Cosine Similarity, computed here as a plain product):

$$\text{Sim}(a, b) = a \times b$$

SoftMax over n raw scores $z_1, \dots, z_n$:

$$\text{SoftMax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Full Worked Example, Step by Step

Step 1: Add Attention Paths from Encoder to Decoder

Attention adds one new connection from every Encoder output directly to every Decoder step, in addition to the single Context Vector connection the basic Encoder-Decoder model already has. No arithmetic yet — this step only changes which values are *available* to the Decoder at each step.

Step 2: Decide What to Compare First

The first thing Attention computes is how similar each Encoder Short-Term Memory is to the *first* Decoder output. Concretely, that means comparing:

- $h_1^{enc} = -0.73$ (encoding of **Let's**) to $h_1^{dec} = -0.99$ (encoding of the Decoder's first input, $\langle \text{EOS} \rangle$)
- $h_2^{enc} = -0.68$ (encoding of **go**) to $h_1^{dec} = -0.99$

No arithmetic yet — this step only sets up which two comparisons are needed.

Step 3: Similarity Score Needed: Let's vs. Decoder Step 1

We need a Similarity Score between the Encoder output for **Let's** and the Decoder's first output, so that the Decoder can later tell how relevant the word **Let's** is at this point in translation.

Step 4: Similarity Score Needed: go vs. Decoder Step 1

Likewise, we need a Similarity Score between the Encoder output for **go** and the Decoder's first output.

Step 5: How to Compute a Similarity Score

With only a single number representing each word (rather than a whole vector), the simplest way to measure similarity is to multiply the two numbers together:

- Same sign $\times$ same sign $\rightarrow$ positive product $\rightarrow$ relatively similar. Example: $2 \times 5 = 10$, $-2 \times -5 = 10$
- Opposite signs $\rightarrow$ negative product $\rightarrow$ relatively dissimilar. Example: $-2 \times 5 = -10$, $2 \times -5 = -10$

(This product is a simplified stand-in for Cosine Similarity; with more than one number per word it generalizes to a Dot Product.)

Step 6: Compute Similarity Score: Let's vs. <EOS>

$$\text{Sim}(h_1^{enc}, h_1^{dec}) = -0.73 \times -0.99 = 0.7227 \approx 0.72$$

The word **Let's** has a Similarity Score of ≈ 0.72 relative to the Decoder's first step.

Step 7: Compute Similarity Score: go vs. <EOS>

$$\text{Sim}(h_2^{enc}, h_1^{dec}) = -0.68 \times -0.99 = 0.6732 \approx 0.67$$

The word **go** has a Similarity Score of ≈ 0.67 relative to the Decoder's first step.

Step 8: Compare the Two Similarity Scores

$$\text{Sim}(\text{Let's}) \approx 0.72 > \text{Sim}(\text{go}) \approx 0.67$$

Because the Similarity Score for **Let's** is higher, we expect **Let's** to end up with more influence over the first word the Decoder generates. No new arithmetic — this step only compares the two results from Steps 6 and 7.

Step 9: Why Run the Similarity Scores Through SoftMax

SoftMax converts the two raw Similarity Scores into two numbers between 0 and 1 that add up to 1, so they can be read as “what percentage of each Encoder output should the Decoder use.”

Step 10: Apply SoftMax to the Similarity Scores

Using the rounded Similarity Scores from Steps 6–7, $z_1 = 0.72$ (Let's) and $z_2 = 0.67$ (go):

$$e^{z_1} = e^{0.72} \approx 2.0544, \quad e^{z_2} = e^{0.67} \approx 1.9542$$

$$\text{sum} = 2.0544 + 1.9542 \approx 4.0087$$

$$\text{SoftMax}(z_1) = \frac{2.0544}{4.0087} \approx 0.5125 \approx 0.51 \quad \text{SoftMax}(z_2) = \frac{1.9542}{4.0087} \approx 0.4875 \approx 0.49$$

The Decoder should use **51%** of the Encoder output for **Let's** and **49%** of the Encoder output for **go** when generating the first output token.

Step 11: Compute the Attention Value (Weighted Sum of Encoder Outputs)

Scale each Encoder Short-Term Memory by its SoftMax weight from Step 10, then add the two scaled values together:

$$-0.73 \times 0.51 = -0.3723 \approx -0.37$$

$$-0.68 \times 0.49 = -0.3332 \approx -0.33$$

$$\text{Attention Value}_1 = -0.37 + (-0.33) = -0.7000 \approx -0.70$$

This Attention Value, ≈ -0.70 , is the single number that summarizes “how much of Let's and how much of go” the Decoder should draw on for its first output token.

Step 12: Fully Connected Layer + SoftMax: Generate the First Output Token

Feed the Attention Value (≈ -0.70) and the Decoder's own encoded value ($h_1^{dec} = -0.99$) into the Fully Connected layer. Its (pretrained, given) raw output — one score per Decoder vocabulary word [ir, Vamos, y, <EOS>] — is:

$$[1.7, 2.1, -3, -1]$$

Run these four raw scores through SoftMax:

$$e^{1.7} \approx 5.4739, \quad e^{2.1} \approx 8.1662, \quad e^{-3} \approx 0.0498, \quad e^{-1} \approx 0.3679$$

$$\text{sum} = 5.4739 + 8.1662 + 0.0498 + 0.3679 \approx 14.0578$$

$$\text{SoftMax} \approx [0.3894, 0.5809, 0.0035, 0.0262]$$

The largest probability, ≈ 0.5809 , is at the position for **Vamos**, so the Decoder outputs the one-hot vector $[0, 1, 0, 0]$ and generates the token **Vamos**.

Step 13: Unroll the Decoder Again, Using Vamos as the Next Input

Since **Vamos** is not the <EOS> token, decoding is not finished. The Decoder LSTM is unrolled for a second step, this time using **Vamos** (one-hot $[0, 1, 0, 0]$) as its input. No new arithmetic yet — this step only sets up the next round, whose Decoder Short-Term Memory output is the given constant $h_2^{dec} = 0.76$.

Step 14: Repeat the Whole Process for the Second Decoder Step

This step repeats Steps 6–12 in full, using $h_2^{dec} = 0.76$ (the encoding of **Vamos**) in place of h_1^{dec} .

Similarity Scores

$$\text{Sim}(h_1^{enc}, h_2^{dec}) = -0.73 \times 0.76 = -0.5548 \approx -0.55$$

$$\text{Sim}(h_2^{enc}, h_2^{dec}) = -0.68 \times 0.76 = -0.5168 \approx -0.52$$

SoftMax of the Similarity Scores

$$e^{-0.55} \approx 0.5769, \quad e^{-0.52} \approx 0.5945, \quad \text{sum} \approx 1.1715$$

$$\text{SoftMax}(-0.55) \approx 0.4925 \approx 0.49 \quad \text{SoftMax}(-0.52) \approx 0.5075 \approx 0.51$$

Attention Value 2

$$-0.73 \times 0.49 = -0.3577 \approx -0.36 \quad -0.68 \times 0.51 = -0.3468 \approx -0.35$$

$$\text{Attention Value}_2 = -0.36 + (-0.35) \approx -0.70$$

Fully Connected Layer + SoftMax

Feed Attention Value₂ (≈ -0.70) and $h_2^{dec} = 0.76$ into the Fully Connected layer. Its given raw output is:

$$[-1, 0, -2, 4.4]$$

$$e^{-1} \approx 0.3679, \quad e^0 \approx 1.0000, \quad e^{-2} \approx 0.1353, \quad e^{4.4} \approx 81.4509$$

$$\text{sum} \approx 82.9541$$

$$\text{SoftMax} \approx [0.0044, 0.0121, 0.0016, 0.9819]$$

The largest probability, ≈ 0.9819 , is at the position for `<EOS>`, so the Decoder outputs the one-hot vector $[0, 0, 0, 1]$ and generates the token `<EOS>`. Since `<EOS>` signals the end of the sentence, decoding is complete: **Let's go** has been translated to **Vamos**.

Step 15: Summary of What Attention Added

No new arithmetic in this step — it summarizes Steps 1–14. Compared to the basic Encoder-Decoder model, the Encoder itself is unchanged. What changed is that at every Decoder step, the Decoder now has direct access to every individual Encoder output (not just a single Context Vector), and uses Similarity Scores plus SoftMax to decide, at that specific step, what percentage of each Encoder output to draw on before making its next prediction.

This completes the full hand-computed Attention example. Every Similarity Score, every SoftMax (including all of its internal exponentials, sums, and divisions), every Attention Value, and every Fully Connected layer decision has been shown with explicit arithmetic, matching all 15 numbered steps of the book's “Attention: Details” section (pages 209–218).